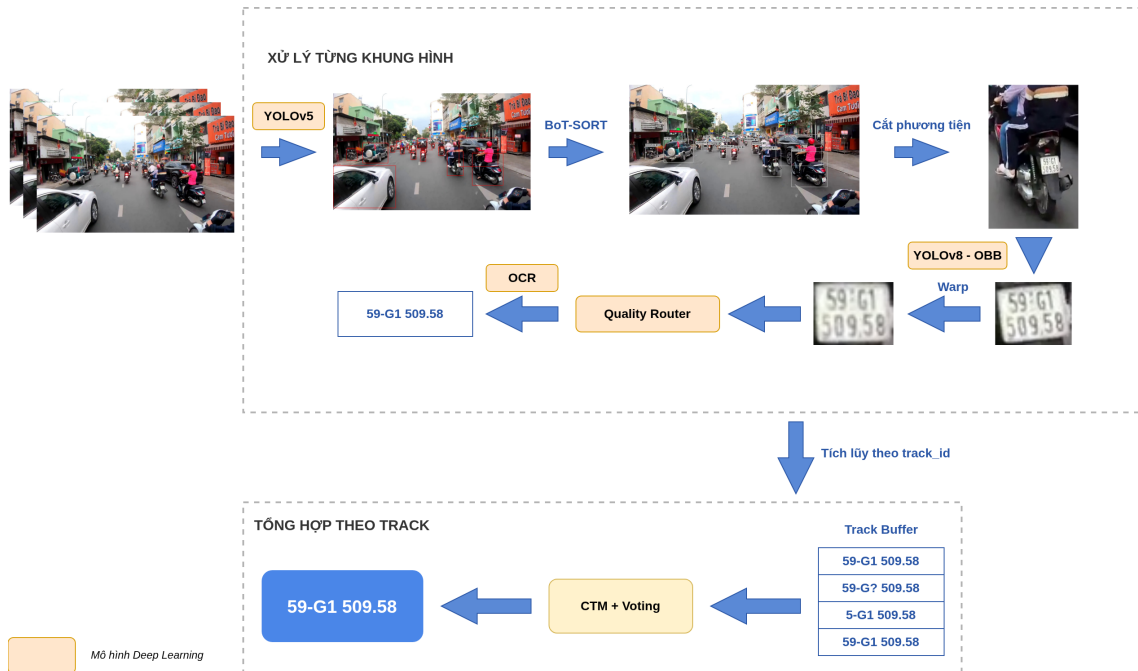


0.1 Tổng quan giải pháp

Pipeline của hệ thống được thiết kế theo hướng xử lý video theo quỹ đạo phương tiện, thay vì nhận dạng biển số độc lập trên từng khung hình. Đầu vào của hệ thống có thể là video tải lên hoặc một đoạn video được trích từ phiên giám sát streaming. Trong cả hai trường hợp, dữ liệu đều được chuẩn hóa thành các chuỗi khung hình; nếu người dùng chọn chế độ tiền xử lý, các phép tăng cường như cải thiện tương phản hoặc giảm nhiễu được áp dụng trước khi đưa khung hình vào các mô hình học sâu. Sơ đồ pipeline của hệ thống được trình bày trong hình 3.1 dưới đây:



Hình 0.1: Sơ đồ tổng quan pipeline xử lý nhận dạng biển số chuyên dùng trên video

Dựa vào Hình 3.1, giải pháp được chia thành sáu khối xử lý chính: Khối phát hiện phương tiện, khối theo vết phương tiện, khối phát hiện biển số, khối phân loại chất lượng ảnh cho OCR, khối OCR và khối tổng hợp đa khung hình. Luồng video đầu vào được chia thành các khung hình và đưa qua khối phát hiện phương tiện trước. Kết quả phát hiện được đưa vào khối theo vết để gán định danh ổn định cho mỗi phương tiện. Sau đó, mỗi phương tiện được theo vết sẽ được cắt vùng và đưa qua khối phát hiện biển số. Biển số sau đó được đưa qua khối phân loại chất lượng để quyết định hướng xử lý. Ảnh biển số sau đó được đem đi nhận dạng ký tự. Cuối cùng, kết quả OCR được tổng hợp đa khung và đưa ra kết quả cuối cùng. Các thành phần của giải pháp:

0.1.1 Khối phát hiện phương tiện

Ở khối này, tôi sử dụng mô hình YOLOv5m để phát hiện các phương tiện giao thông phổ biến trong bối cảnh Việt Nam: xe máy, xe con, xe tải, xe tải van, xe buýt

và xe ba gác chở hàng. Trọng số của mô hình được lấy trực tiếp từ công trình của Che et al.[1], đã được huấn luyện trên dữ liệu phương tiện thu thập trong điều kiện giao thông Việt Nam, do đó mô hình có khả năng thích nghi tốt với đặc thù mật độ xe, góc quay camera và điều kiện ánh sáng phổ biến tại Việt Nam.

0.1.2 Khối theo vết phương tiện

Sau khi phát hiện, các hộp bao phương tiện được đưa vào thuật toán BoT-SORT để gán và duy trì `track_id` ổn định xuyên suốt các khung hình. Việc đặt bước phát hiện phương tiện ở đầu pipeline giúp giảm vùng tìm kiếm cho biển số, đồng thời tạo cơ sở để gom nhiều quan sát biển số của cùng một xe vào một track thống nhất. Quyết định theo vết phương tiện thay vì theo vết biển số trực tiếp dựa trên nhận định rằng phương tiện lớn hơn biển số, xuất hiện ổn định hơn và có nhiều đặc trưng ngoại hình để BoT-SORT duy trì định danh. Biển số nhỏ, dễ mất nét, bị lóa hoặc bị che; nếu tracking trực tiếp biển số, chỉ vài khung hình xấu đã có thể làm đứt quỹ đạo. Vì vậy, biển số được xem như quan sát gắn vào track phương tiện.

0.1.3 Khối phát hiện biển số

Với các phương tiện đang được theo vết, hệ thống cắt vùng ảnh phương tiện và mở rộng thêm biên 8% để tránh mất biển số ở đầu hoặc đuôi xe, sau đó chạy mô hình YOLOv8-OBB trên từng vùng được cắt. Lý do sử dụng hộp bao có hướng (oriented bounding box) thay vì hộp bao ngang (horizontal bounding box) là biển số xe trong điều kiện thực tế thường bị nghiêng do góc quay camera, hướng di chuyển của phương tiện hoặc phối cảnh đường phố. OBB giúp vùng biển số được bao sát hơn, giảm phần nền thừa khi crop và tạo đầu vào ổn định hơn cho bước OCR phía sau.

0.1.4 Khối phân loại chất lượng ảnh cho OCR

Sau khi được phát hiện, mỗi biển số được đưa qua bộ phân loại chất lượng trước khi quyết định có nên OCR ngay hay không. Mục tiêu của bộ định tuyến này là phân tách các ảnh biển số thành ba nhánh xử lý khác nhau: *direct* — ảnh đủ rõ để OCR ngay trong khung hình hiện tại; *tracklet_fusion* — ảnh chưa đủ chất lượng, được đưa vào bộ đệm track để tổng hợp với các khung hình khác; và *unreadable_wait* — ảnh quá mờ hoặc bị che khuất, chỉ được lưu làm bằng chứng mà không chạy OCR. Việc tách riêng bước này giúp hệ thống không ép mô hình OCR đưa ra kết luận cuối cùng từ một khung hình xấu, đồng thời được thiết kế nhằm hạn chế số lần OCR không cần thiết.

0.1.5 Khối OCR

Khối OCR thực hiện nhận dạng ký tự trên ảnh biển số đã được cắt và định tuyến qua khối phân loại chất lượng. Đầu vào của khối là ảnh cắt biển số đã được nắn

chỉnh phối cảnh và đưa về kích thước chuẩn. Đầu ra là chuỗi ký tự biển số kèm theo xác suất cho từng ký tự.

Lý do lựa chọn mô hình nhận dạng biển số chuyên biệt thay vì sử dụng các mô hình OCR lớn có sẵn dựa trên ba nhận định:

Thứ nhất, các mô hình OCR lớn được thiết kế để nhận dạng tất cả các loại ký tự trong văn bản tự do, nên chứa rất nhiều tham số không cần thiết cho bài toán biển số. Việc sử dụng nguyên mô hình lớn đó trong hệ thống sẽ gây lãng phí tài nguyên tính toán, làm tăng đáng kể độ trễ suy luận và tiêu tốn bộ nhớ.

Thứ hai, biển số xe có đặc thù rất riêng: chỉ bao gồm các chữ số và chữ in hoa, được sắp xếp theo khoảng cách và bố cục cố định, khác xa văn bản thông thường với đa dạng phong chữ, cỡ chữ và cách sắp xếp. Mô hình OCR tổng quát không tận dụng được các ràng buộc này và dễ đưa ra dự đoán sai lệch do học từ quá nhiều lớp ký tự không liên quan.

Thứ ba, với tập dữ liệu biển số đã thu thập được, việc tự huấn luyện một mô hình nhỏ gọn chuyên biệt cho bài toán này là khả thi và mang lại hiệu quả vượt trội: mô hình có kích thước nhỏ hơn nhiều, chạy nhanh hơn, tiêu tốn ít tài nguyên hơn, đồng thời được tối ưu hóa trực tiếp cho đặc thù biển số Việt Nam.

Hệ thống sử dụng mô hình `SmallLPR-Line-CTC` làm bộ nhận dạng biển số. Mô hình này là biến thể cải tiến của `LPRNet`, được thiết kế riêng cho đặc thù biển số Việt Nam với bốn cải tiến chính: `STN` (`Spatial Transformer Network`) để bù sai lệch hình học, khối `SmallBasicBlockCBAM` kết hợp `MixConv` và `CBAM attention` để trích xuất đặc trưng đa tỷ lệ, giữ `feature map` hai chiều kết hợp `positional encoding` hai chiều để bảo toàn thông tin vị trí theo hàng và cột, và kiến trúc `multi-head CTC` với `layout_head` để định tuyến giải mã riêng cho biển một dòng và hai dòng.

0.1.6 Khối tổng hợp đa khung hình

Khi một track phương tiện kết thúc — tức phương tiện biến mất trong một số bước liên tiếp hoặc video đã được xử lý đến khung hình cuối cùng — hệ thống tiến hành tổng hợp toàn bộ bằng chứng nhận dạng ký tự đã thu thập được trong suốt thời gian tồn tại của track.

Thay vì biểu quyết trực tiếp trên toàn bộ chuỗi biển số, hệ thống thực hiện biểu quyết độc lập theo từng vị trí ký tự. Với mỗi vị trí, hệ thống chọn ký tự có tổng số phiếu cao nhất, nhưng chỉ chấp nhận nếu đồng thời thỏa mãn hai điều kiện: tỷ lệ phiếu đạt ít nhất 0,5 và xác suất trung bình của ký tự đạt ít nhất 0,50. Những vị trí không thỏa mãn được gán ký hiệu “?” và track bị từ chối nếu còn tồn tại vị trí chưa

xác định. Cách tiếp cận này cho phép giữ lại các ký tự đúng ngay cả khi một số vị trí khác bị nhiễu, đồng thời ngăn chặn kết quả sai lệch từ các khung hình xấu thông qua ngưỡng đồng thuận.

0.2 Các mô hình và thuật toán

0.2.1 Thuật toán theo vết phương tiện

Hệ thống xử lý video dựa trên quỹ đạo phương tiện thay vì nhận dạng đơn lẻ từng khung hình. Lỗi theo vết gồm hai bước chính: (1) theo vết phương tiện để cấp định danh (v_tid), và (2) phát hiện nối tiếp (cascade detection) để liên kết biển số vào phương tiện.

Theo vết phương tiện với BoT-SORT. Pipeline sử dụng thuật toán BoT-SORT [2] để gán và duy trì định danh xuyên suốt cho mỗi phương tiện. Khung hình đầu vào trước tiên được xử lý bởi mô hình YOLOv5m nhằm phát hiện sáu lớp phương tiện phổ biến tại Việt Nam. Các hộp bao tìm được sẽ được BoT-SORT xử lý qua bộ lọc Kalman và thuật toán ghép cặp Hungarian để theo vết.

Việc theo vết được áp dụng trên phương tiện thay vì biển số nhằm tăng độ ổn định. Phương tiện có kích thước lớn và đặc trưng ngoại hình rõ ràng, giúp BoT-SORT dễ dàng duy trì quỹ đạo ngay cả khi bị che khuất một phần. Biển số, ngược lại, chỉ được xem là một thuộc tính phụ thuộc của phương tiện.

Phát hiện tiếp nối và liên kết không gian. Để định vị biển số, hệ thống áp dụng kỹ thuật phát hiện tiếp nối (Cascade Detection). Với mỗi phương tiện đang được theo vết, vùng ảnh chứa phương tiện đó được cắt và mở rộng lên 8% trước khi đưa vào mô hình YOLOv8-OBB. Đặc tính hình học của phương pháp này cho phép liên kết trực tiếp: biển số phát hiện trên vùng ảnh của phương tiện nào sẽ mặc định được gán cho v_tid của phương tiện đó.

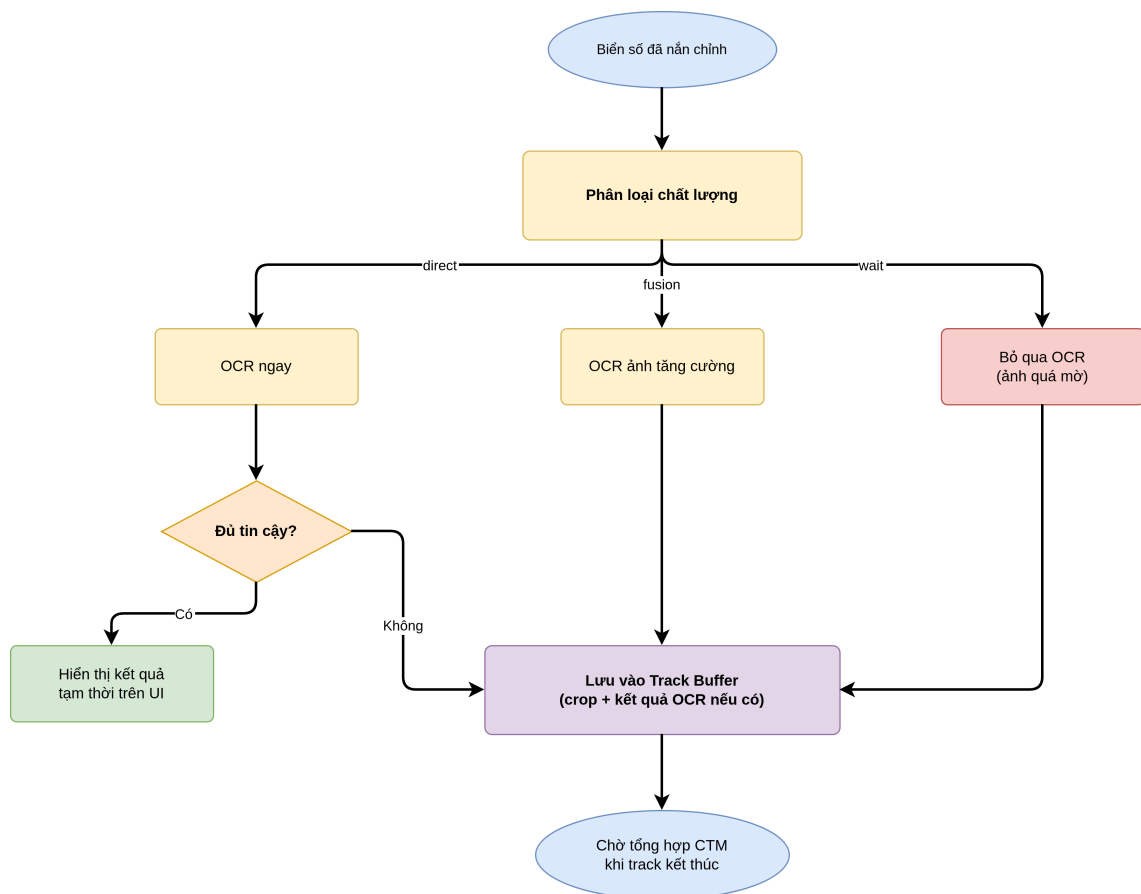
Trong trường hợp các phương tiện di chuyển sát nhau (ví dụ xe máy che khuất một phần ô tô), vùng ảnh cắt của một xe có thể vô tình chứa biển số của xe khác. Do các vùng cắt phương tiện có thể chồng lấp, các OBB biển số được ánh xạ về hệ tọa độ khung hình gốc trước khi liên kết. Các phát hiện trùng lặp được khử theo mức chồng lấp hình học; ứng viên còn lại được gán cho track có hộp bao gốc chứa tâm biển số. Nếu tâm biển số đồng thời nằm trong nhiều hộp bao phương tiện do hiện tượng chồng lấp, hệ thống chọn track có diện tích hộp bao nhỏ nhất trong tập các hộp bao chứa tâm biển số. Quy tắc này bảo đảm mỗi biển số chỉ được liên kết với một track duy nhất.

Tóm lại. Quy trình theo vết và ghép cặp bao gồm:

1. **Phát hiện phương tiện:** Phân loại và lấy hộp bao phương tiện bằng YOLOv5.
2. **Theo vết đối tượng:** BoT-SORT cấp phát và duy trì v_tid cho từng hộp bao.
3. **Cắt ảnh (Crop):** Cắt vùng ảnh của từng phương tiện đã được gán ID.
4. **Phát hiện biển số:** Dùng YOLOv8-OBB tìm biển số trên các vùng ảnh cắt.
5. **Ghép cặp (Association):** Ánh xạ OBB biển số về khung hình gốc, liên kết với v_tid có hộp bao chứa tâm biển số, ưu tiên hộp bao nhỏ nhất khi có chồng lấp.

0.2.2 Bộ định tuyến chất lượng (Quality Router)

Sau khi biển số được trích xuất, hệ thống không đưa trực tiếp vào OCR mà chạy qua một bộ định tuyến chất lượng (quality router) trước. Không phải mọi khung hình đều cho ảnh biển số đủ rõ — một số bị mờ chuyển động, lóa đèn, hoặc bị che khuất một phần. Nếu các khung hình này vẫn được tính vào cơ chế biểu quyết, chúng sẽ đóng góp phiếu bầu sai và làm giảm độ tin cậy của kết quả cuối. Do đó, chỉ các biển số vượt ngưỡng chất lượng mới được đưa vào OCR và tính phiếu bầu; các biển số bị loại được bỏ qua hoàn toàn. Quy trình phân loại chất lượng và định tuyến xử lý được mô tả bằng lưu đồ trong Hình 0.2.



Hình 0.2: Lưu đồ thuật toán phân loại chất lượng và định tuyến xử lý

Cơ chế định tuyến bằng YOLOv8n-cls. Ảnh biến số sau khi nắn chỉnh phối cảnh được đưa qua mạng phân loại YOLOv8n-cls để dự đoán mức độ đọc được theo bốn lớp: *perfect*, *good*, *poor* và *illegible*. Dựa trên các nhãn này, hệ thống rẽ nhánh luồng xử lý:

- **Direct (*perfect*, *good*):** Ảnh đủ sắc nét, được đưa thẳng vào mô hình OCR để ưu tiên phát ra kết quả sớm.
- **Tracklet Fusion (*poor*):** Ảnh suy giảm chất lượng nhưng vẫn chứa thông tin cấu trúc, được đưa vào bộ đệm chờ bỏ phiếu đa khung hình. Trước khi OCR, các ảnh trong bộ đệm được tăng cường chất lượng bằng các phép xử lý ảnh: Unsharp Masking làm sắc nét ảnh bị mờ chuyển động, Gamma Correction và CLAHE tăng tương phản cục bộ cho ảnh thiếu sáng hoặc chói sáng, và Bilateral Filter giảm nhiễu hạt trong khi bảo toàn cạnh chữ. Các phiên bản ảnh tăng cường này được OCR độc lập, sau đó hệ thống chọn ứng viên tốt nhất dựa trên xác suất ký tự và chi phí sửa lỗi để đưa vào biểu quyết đa khung hình.
- **Unreadable Wait (*illegible*):** Ảnh bị nhòe hoàn toàn hoặc che khuất nặng, chỉ lưu trữ làm bằng chứng hình ảnh và bỏ qua hoàn toàn bước OCR.

Điểm chất lượng (q-score). Bên cạnh nhãn phân loại, hệ thống tính toán thêm một điểm chất lượng liên tục $q \in [0, 1]$ bằng thuật toán xử lý ảnh truyền thống. Điểm q kết hợp độ sắc nét (dựa trên toán tử không gian) và kích thước của biến số:

$$q = 0.80 \times \min \left(\frac{\text{Var}(\nabla^2 I)}{500}, 1 \right) + 0.20 \times \min \left(\frac{h \times w}{A_{\text{ref}}}, 1 \right)$$

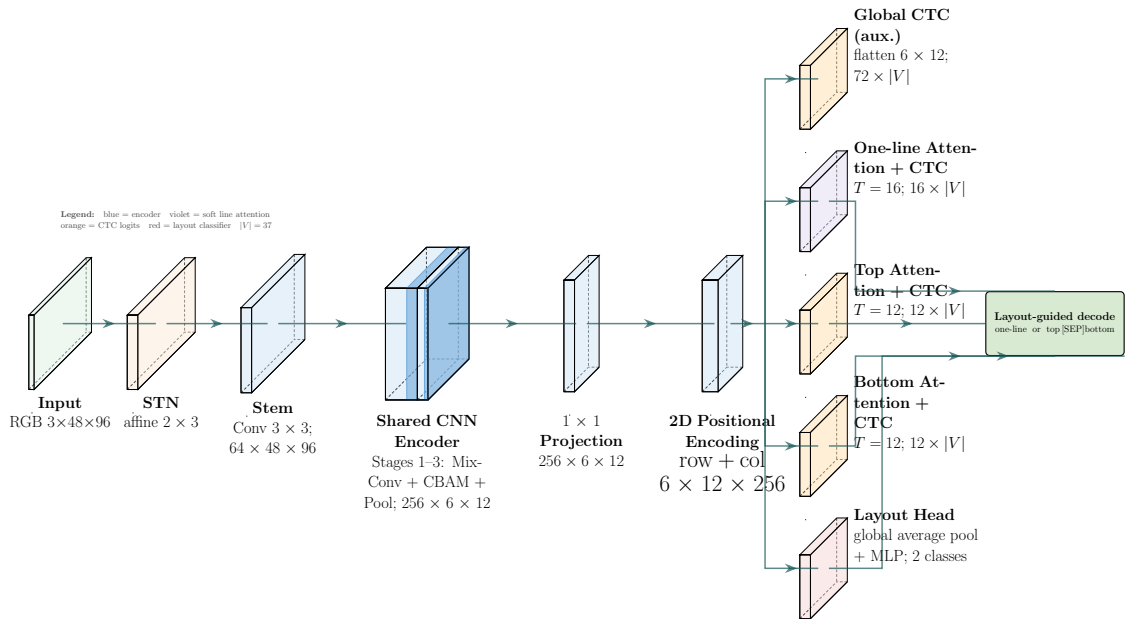
Cần lưu ý rằng, **điểm q này không can thiệp vào quyết định định tuyến chính của mạng Deep Learning**. Bất cứ khi nào mạng phân loại dự đoán ảnh là *perfect* hoặc *good*, ảnh đó sẽ luôn được đưa đi OCR trực tiếp (nhánh Direct) bất kể điểm q được tính ra cao hay thấp.

Khi mạng Deep Learning đang hoạt động bình thường, điểm q chỉ đóng vai trò **xếp hạng trong bộ đệm (Buffer Ranking)**. Cụ thể, đối với các ảnh mờ bị đẩy vào nhánh *Tracklet Fusion*, điểm q sẽ đóng vai trò như một trọng số chất lượng để giúp hệ thống chọn ra khung hình tốt nhất làm đại diện khi biểu quyết đa khung hình. Chỉ khi mạng Deep Learning bị vô hiệu hóa, điểm q mới được kích hoạt chế độ dự phòng (fallback) để tự động thay thế nhiệm vụ rẽ nhánh.

0.2.3 Kiến trúc và thiết kế mô hình OCR (SmallLPR-Line-CTC)

Hạn chế của LPRNet gốc với bài toán biển số Việt Nam. Khối OCR được xây dựng từ tinh thần LPRNet: nhận dạng trực tiếp toàn bộ ảnh biển số bằng CNN nhẹ kết hợp CTC, không cần tách ký tự thủ công [3]. Khi áp dụng cho biển số Việt Nam, LPRNet gốc bộc lộ bốn hạn chế chính: (1) ảnh biển số được cắt từ video thường bị nghiêng hoặc lệch phối cảnh, trong khi mô hình gốc không có cơ chế căn chỉnh hình học; (2) các khối tích chập nhẹ chưa khai thác tốt nét ký tự nhỏ, nhiễu cục bộ và ngữ cảnh rộng; (3) đọc đặc trưng như chuỗi một chiều làm suy giảm thông tin vị trí theo chiều dọc, cần thiết cho biển hai dòng; (4) một head CTC duy nhất buộc mọi bố cục đi qua cùng một dòng giải mã, gây đọc sai thứ tự ở biển hai dòng.

Tổng quan SmallLPR-Line-CTC. Để giải quyết bốn hạn chế trên, tôi đề xuất mô hình SmallLPR-Line-CTC. Mô hình có khoảng 1,5 triệu tham số, nhờ đó giữ được quy mô nhỏ gọn và phù hợp với yêu cầu suy luận lặp lại trên nhiều khung hình trong pipeline video. Kiến trúc tổng thể của mô hình được trình bày trong Hình 0.3.



Hình 0.3: Kiến trúc mô hình OCR SmallLPR-Line-CTC

STN để bù sai lệch hình học. STN đặt ở đầu mô hình dự đoán ma trận affine 2×3 (sáu tham số) để biến đổi ảnh biển số trước khi đưa vào backbone. Tầng FC cuối được khởi tạo thành phép biến đổi đồng nhất, nên ở đầu huấn luyện mô hình không làm biến dạng ảnh; khi dữ liệu cho thấy dịch chuyển, xoay hoặc co giãn nhẹ giúp giảm CTC loss, STN tự học phép hiệu chỉnh tương ứng.

Bảng 1: Kiến trúc STN alignment network

Layer	Tham số	Kích thước đầu ra
Input	RGB image	$3 \times 48 \times 96$
Conv	$3 \times 3, 3 \rightarrow 32$	$32 \times 46 \times 94$
MaxPool + Mish	2×2	$32 \times 23 \times 47$
Conv	$5 \times 5, 32 \rightarrow 32$	$32 \times 19 \times 43$
MaxPool + Mish	3×3	$32 \times 6 \times 14$
FC + Mish	$2688 \rightarrow 32$	32
FC	$32 \rightarrow 6$	affine 2×3

SmallBasicBlockCBAM để tăng năng lực trích đặc trưng. Sau khi trích đặc trưng đa tỷ lệ bằng MixConv, CBAM được thêm vào cuối block để tái trọng số đặc trưng theo kênh và theo không gian [4]. Attention theo kênh giúp chọn nhóm đặc trưng hữu ích cho nét ký tự, còn attention theo không gian tăng trọng số các vùng chứa ký tự so với nền, vít, khung biển hoặc nhiễu do crop. Backbone vẫn giữ tinh thần nhỏ gọn của LPRNet nhưng tăng khả năng phân biệt chi tiết ký tự trong điều kiện video không ràng buộc.

Bảng 2: Khối cơ bản SmallBasicBlockCBAM

Thành phần	Mô tả
MixConv	Trích xuất đặc trưng bằng nhiều kernel, $C_{in} \rightarrow C_{out}/4$
Conv 3×1	Khai thác nét dọc của ký tự
Conv 1×3	Khai thác nét ngang và quan hệ ký tự theo chiều biển
Conv 1×1	Khôi phục số kênh về C_{out}
BatchNorm + Mish	Ổn định huấn luyện và tăng phi tuyến
CBAM	Tái trọng số đặc trưng theo kênh và không gian
Residual	Shortcut connection

Feature map hai chiều và positional encoding. Mô hình giữ đặc trưng ở dạng 6×12 sau backbone thay vì làm phẳng ngay thành chuỗi một chiều. Sau lớp projection 1×1 , mỗi vị trí không gian được cộng mã hóa vị trí hai chiều gồm embedding theo hàng và embedding theo cột. Biểu diễn này cho phép mô hình biết một đặc trưng nằm ở dòng nào và cột nào trên biển số, nhưng vẫn giữ khả năng học mềm thay vì áp đặt quy tắc tách dòng cố định. Đây là nền tảng để các nhánh attention phía sau gom đặc trưng theo một dòng hoặc hai dòng mà không cần detector dòng riêng.

Trong đó, $|V| = 37$ là kích thước tập từ vựng đầu ra, bao gồm 10 chữ số, 23 chữ cái, 3 ký tự phân cách ($-$, $.$, $[SEP]$) và 1 ký tự rỗng ($\langle blank \rangle$) phục vụ cho giải

Bảng 3: Backbone CNN của SmallLPR-Line-CTC

Layer	Tham số	Kích thước đầu ra
Input	RGB image	$3 \times 48 \times 96$
Stem	Conv 3×3 , $3 \rightarrow 64$ + BN + Mish	$64 \times 48 \times 96$
Stage 1	SmallBasicBlockCBAM(64, 128) + MaxPool2	$128 \times 24 \times 48$
Stage 2	SmallBasicBlockCBAM(128, 256)	$256 \times 24 \times 48$
	SmallBasicBlockCBAM(256, 256) + MaxPool2	$256 \times 12 \times 24$
Stage 3	SmallBasicBlockCBAM(256, 256)	$256 \times 12 \times 24$
	SmallBasicBlockCBAM(256, 256) + MaxPool2	$256 \times 6 \times 12$
Projection 2D positional encoding	Conv 1×1 , $256 \rightarrow 256$	$256 \times 6 \times 12$
	row embedding + column embedding	$6 \times 12 \times 256$

Bảng 4: Các head nhận dạng và bố cục

Head	Đầu vào	Đầu ra
global_head	flatten 6×12	$72 \times V $
one_line_attention	$6 \times 12 \times 256$	attention 6×12
one_line_head	sequence 16 bước	$16 \times V $
line_attention	$6 \times 12 \times 256$	2 attention maps
top_head	sequence 12 bước	$12 \times V $
bottom_head	sequence 12 bước	$12 \times V $
layout_head	global avg pooled	2 logits

thuật CTC.

Layout-aware multi-head CTC. Thay vì một head CTC duy nhất, *SmallLPR-Line-CTC* sử dụng năm head song song hoạt động trên cùng feature map $6 \times 12 \times 256$. `global_head` flatten toàn bộ feature map thành chuỗi 72 bước. `one_line_head` nhận chuỗi 16 bước được gom bởi `one_line_attention`. `top_head` và `bottom_head` mỗi head nhận chuỗi 12 bước được tách bởi `line_attention`, trong đó attention dòng trên được khởi tạo với prior tập trung vào nửa trên feature map. `layout_head` phân loại bố cục từ global average pooling, đầu ra 2 logit.

Ở giai đoạn suy luận, nếu xác suất bố cục hai dòng từ `layout_head` vượt ngưỡng 0.5, hệ thống giải mã `top_head` và `bottom_head` rồi nối bằng token `[SEP]`; ngược lại dùng `one_line_head`.

Hàm mất mát huấn luyện. Vì kiến trúc có nhiều head theo bố cục, hàm mất mát tổng hợp gồm CTC loss cho các nhánh nhận dạng và cross-entropy loss cho `layout_head`. Loss được mask theo bố cục: biến một dòng chỉ đóng góp vào `one_line_head`, biến hai dòng vào `top_head` và `bottom_head`, còn `global_head` và `layout_head` huấn luyện trên cả hai loại.

$$\mathcal{L} = \lambda_g \mathcal{L}_{global} + \lambda_1 \mathcal{L}_{one} + \lambda_t \mathcal{L}_{top} + \lambda_b \mathcal{L}_{bottom} + \lambda_l \mathcal{L}_{layout}$$

Trong đó $\mathcal{L}_{layout}$ là cross-entropy loss phân loại một dòng/hai dòng. Trọng số huấn luyện: $\lambda_g = \lambda_1 = \lambda_t = \lambda_b = 1.0$, $\lambda_l = 0.2$. Trọng số nhỏ cho $\mathcal{L}_{layout}$ phản ánh vai trò phụ trợ của nhánh bố cục — nhiệm vụ chính vẫn là nhận dạng chính xác chuỗi ký tự. CTC loss được dùng vì nhãn huấn luyện chỉ có chuỗi ký tự biến số, không cần annotation vị trí ký tự. CTC tự động căn hàng giữa chuỗi dự đoán và nhãn bằng cách cho phép token rỗng `<blank>` và token lặp, giúp mô hình học end-to-end mà không cần tách ký tự thủ công.

0.3 Xử lý kết quả sau mô hình

Đầu ra thô từ mô hình OCR là một chuỗi các cặp (ký tự, xác suất) $\mathcal{P} = \{(c_1, p_1), (c_2, p_2), \dots, (c_L, p_L)\}$. Đầu ra này không thể sử dụng trực tiếp do: (1) nhiễu nhận diện giữa các ký tự tương đồng, (2) độ tin cậy của một khung hình đơn lẻ thường thấp, và (3) chưa chắc chắn tuân thủ quy tắc biến số hợp lệ. Mục này mô tả hai module hậu xử lý cốt lõi: Xác thực định dạng và Tổng hợp đa khung hình (CTM).

0.3.1 Xác thực định dạng biến số

1. Chuẩn hóa và rà soát định dạng Chuỗi ký tự thô được chuẩn hóa thông qua các thao tác:

- Chuyển đổi toàn bộ ký tự sang chữ in hoa.
- Loại bỏ khoảng trắng thừa và chuẩn hóa ký hiệu phân dòng (đối với biển hai dòng) thành dấu cách ngang.

Chuỗi sau chuẩn hóa được đối chiếu với tập K mẫu định dạng (Templates) hợp lệ của biển số Việt Nam (ví dụ: DDL-DDD.DD, DDLL-DDDD). Mỗi mẫu định nghĩa ràng buộc chặt chẽ cho từng vị trí (slot): chữ số (D), chữ cái (L), hoặc hỗn hợp (A).

2. Sửa lỗi ký tự mơ hồ (Slot-aware Error Correction) Mô hình OCR thường nhầm lẫn các cặp ký tự có hình thái tương đồng trong tập ánh xạ hai chiều

$$\mathcal{M} = \{O/0, I/1, S/5, B/8, G/6, Z/2\}.$$

Việc thay thế không được thực hiện trên toàn chuỗi, vì cùng một ký tự có thể đúng ở vị trí này nhưng sai ở vị trí khác. Chẳng hạn, chữ B phải được giữ nguyên tại slot chữ cái L , nhưng có thể được sửa thành số 8 tại slot chữ số D .

Gọi $\mathbf{x} = (x_1, x_2, \dots, x_m)$ là chuỗi OCR sau chuẩn hóa và $\mathcal{T} = (t_1, t_2, \dots, t_n)$ là một mẫu biển số. Mỗi t_i là slot chữ số D , slot chữ cái L , slot hỗn hợp A , hoặc một ký tự phân cách cố định như “-”, “.” và [SEP]. Ký hiệu $x_j \models t_i$ cho biết x_j hợp lệ tại vị trí t_i : chữ số hợp lệ với D , chữ cái hợp lệ với L , chữ hoặc số hợp lệ với A , còn ký tự phân cách phải trùng chính xác. Chi phí căn chỉnh được trình bày trong Bảng 5.

Bảng 5: Chi phí căn một token OCR vào một vị trí của mẫu

Trường hợp	Điều kiện	c_{align}
Match slot	$t_i \in \{D, L, A\}$ và $x_j \models t_i$	0
Replace mơ hồ	$x_j \not\models t_i$, nhưng ánh xạ trong $\mathcal{M}$ tạo ra ký tự hợp lệ	0.12
Sai loại slot	x_j là chữ hoặc số, nhưng không có ánh xạ mơ hồ phù hợp	2.0
Token ngoài slot	Cần token phân cách vào một slot D, L hoặc A	2.4
Match ký tự cố định	$t_i \notin \{D, L, A\}$ và $x_j = t_i$	0
Sai ký tự cố định	$t_i \notin \{D, L, A\}$ và $x_j \neq t_i$	2.0

Như vậy, match có chi phí 0 khi token đã đúng loại slot hoặc đúng ký tự phân cách. Replace chỉ xảy ra khi token sai loại nhưng có ánh xạ mơ hồ làm nó trở thành hợp lệ, ví dụ $O \rightarrow 0$ tại slot D hoặc $0 \rightarrow O$ tại slot L ; trường hợp này có chi phí

0.12. Hai thao tác còn lại có chi phí phụ thuộc loại token:

$$c_{\text{missing}}(t_i) = \begin{cases} 1.4, & t_i \in \{D, L, A\}, \\ 1.0, & t_i = [\text{SEP}], \\ 0.75, & t_i \in \{-, \cdot\}, \end{cases} \quad (1)$$

$$c_{\text{skip}}(x_j) = \begin{cases} 1.1, & x_j \text{ là chữ hoặc số}, \\ 0.7, & x_j \text{ là token phân cách}. \end{cases}$$

Chi phí nhỏ hơn cho dấu phân cách phản ánh việc OCR bỏ sót hoặc nhận dư dấu “-”, “.” và $[\text{SEP}]$ ít nghiêm trọng hơn thiếu hoặc thừa một ký tự thuộc nội dung biểu số.

Đặt $DP[i, j]$ là chi phí nhỏ nhất để căn j token đầu của chuỗi OCR vào i vị trí đầu của mẫu. Điều kiện biên được khởi tạo bằng cách tích lũy chi phí thiếu vị trí mẫu hoặc bỏ qua token đầu vào:

$$\begin{aligned} DP[0, 0] &= 0, \\ DP[i, 0] &= DP[i - 1, 0] + c_{\text{missing}}(t_i), \\ DP[0, j] &= DP[0, j - 1] + c_{\text{skip}}(x_j). \end{aligned} \quad (2)$$

Với $i > 0$ và $j > 0$, mỗi ô được tính từ ba trạng thái tiền nhiệm:

$$DP[i, j] = \min \begin{cases} DP[i - 1, j - 1] + c_{\text{align}}(x_j, t_i), & \text{align}, \\ DP[i - 1, j] + c_{\text{missing}}(t_i), & \text{missing}, \\ DP[i, j - 1] + c_{\text{skip}}(x_j), & \text{skip}. \end{cases} \quad (3)$$

Các ô được tính lần lượt từ trái sang phải, từ trên xuống dưới. Sau khi điền xong bảng, hệ thống truy vết ngược từ $DP[n, m]$ về $DP[0, 0]$. Nhánh *align* tiêu thụ đồng thời một token OCR và một vị trí mẫu; nhánh *missing* chỉ tiêu thụ vị trí mẫu và đánh dấu vị trí đó bị thiếu; nhánh *skip* chỉ tiêu thụ token OCR và loại token nhận diện dư.

Quy trình trên được thực hiện độc lập với từng mẫu $\mathcal{T}_k$ trong tập K mẫu. Gọi r_k là số phép replace khi căn theo mẫu $\mathcal{T}_k$, điểm của mẫu và quy tắc chọn mẫu cuối cùng là

$$C(\mathcal{T}_k) = DP_k[n_k, m] + 0.055r_k, \quad \mathcal{T}^* = \arg \min_{\mathcal{T}_k} C(\mathcal{T}_k). \quad (4)$$

Nếu nhiều mẫu có cùng điểm, hệ thống ưu tiên mẫu cần ít phép replace hơn. Ví dụ,

khi căn chuỗi 30G-51B.27 với mẫu DDL-DDD.DD, ký tự G đã hợp lệ tại slot L nên có chi phí 0, còn B nằm tại slot D được thay bằng 8 với chi phí 0.12. Sau khi cộng mức phạt rủi ro 0.055 cho một phép replace, tổng phần chi phí sửa là 0.175 và kết quả thu được là 30G-518.27. Xác suất của ký tự bị thay thế được giảm theo hệ số $\gamma = 0.92$, tức $p'_j = p_j \times \gamma$; các ký tự match giữ nguyên xác suất ban đầu.

0.3.2 Tổng hợp đa khung hình

1. Tiền xử lý và chọn lọc dữ liệu (Buffer Selection) Các khung hình không được chốt ngay (do thuộc nhóm *tracklet_fusion* hoặc thuộc nhóm *direct* nhưng độ tin cậy $p < 0.90$) sẽ được đưa vào bộ đệm (Track Buffer). Hệ thống thực hiện tiền xử lý tăng cường để tạo ra các ứng viên (candidates) tốt hơn:

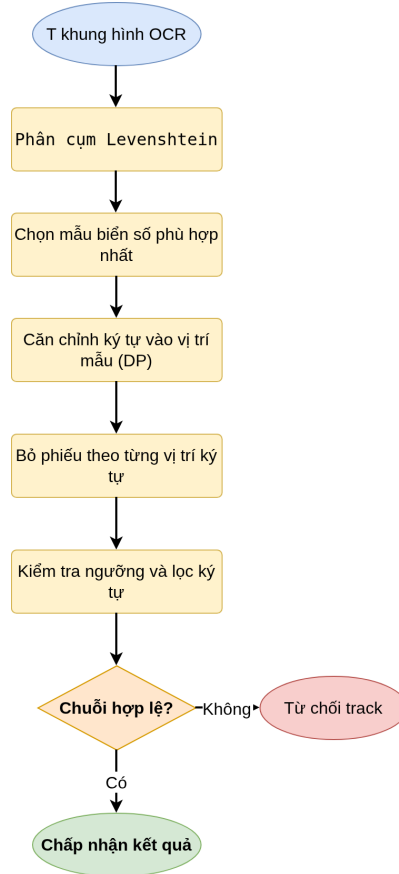
- **Unsharp Masking:** Làm sắc nét cho ảnh bị mờ nhòe (motion blur).
- **Gamma Correction & CLAHE:** Tăng cường tương phản cục bộ trên kênh độ sáng L (không gian màu LAB) cho ảnh thiếu sáng hoặc chói sáng.
- **Bilateral Filter:** Giảm nhiễu hạt và sương mù trong khi vẫn bảo toàn cạnh ảnh.

Các phiên bản ảnh này được chạy OCR độc lập. Ứng viên tốt nhất được chọn thông qua hàm xếp hạng (Rerank) kết hợp xác suất ký tự và chi phí sửa lỗi, sau đó mới được lưu vào bộ đệm.

2. Cơ chế Character Time-series Matching (CTM) Khi một track kết thúc, hệ thống kích hoạt CTM trên các kết quả OCR đã được chọn từ bộ đệm. Tổng quan cơ chế CTM như sau:

Trước khi hợp nhất, hệ thống tạo một chuỗi dùng riêng cho việc so sánh bằng cách loại các dấu phân cách; chuỗi token đầy đủ vẫn được giữ lại cho bước căn chỉnh. Các kết quả OCR được phân cụm theo độ tương đồng Levenshtein chuẩn hóa $1 - d_{\text{Lev}}(a, b) / \max(|a|, |b|)$. Hai chuỗi được đưa vào cùng cụm nếu độ tương đồng đạt ít nhất 0.6; số cụm được giới hạn tối đa là 3. Bước này hạn chế việc kết quả của nhiều biển số bị trộn vào cùng một lần biểu quyết khi định danh theo vết bị tái sử dụng. CTM được thực hiện độc lập trên từng cụm và cụm hợp lệ có nhiều quan sát nhất được chọn làm kết quả chính.

Chọn mẫu và căn chỉnh bằng quy hoạch động. Xét một cụm gồm N chuỗi OCR. Chuỗi thứ r được ký hiệu $\mathbf{x}^{(r)} = (x_1^{(r)}, x_2^{(r)}, \dots, x_{m_r}^{(r)})$, trong đó mỗi token đi kèm xác suất nhận dạng. Với mẫu biển số $\mathcal{T}_k = (t_1^{(k)}, t_2^{(k)}, \dots, t_{n_k}^{(k)})$, đặt $D_{r,k}[i, j]$ là chi phí nhỏ nhất để căn j token đầu của chuỗi thứ r vào i vị trí đầu của mẫu. Bảng quy



Hình 0.4: Cơ chế tổng hợp đa khung hình CTM

hoạch động được khởi tạo bởi

$$\begin{aligned}
 D_{r,k}[0, 0] &= 0, \\
 D_{r,k}[i, 0] &= D_{r,k}[i - 1, 0] + c_{\text{missing}}^{\text{CTM}}(t_i^{(k)}), \\
 D_{r,k}[0, j] &= D_{r,k}[0, j - 1] + c_{\text{skip}}^{\text{CTM}}(x_j^{(r)}).
 \end{aligned} \tag{5}$$

Với $i > 0$ và $j > 0$, mỗi ô được tính từ ba trạng thái tiền nhiệm:

$$D_{r,k}[i, j] = \min \begin{cases} D_{r,k}[i - 1, j - 1] + c_{\text{align}}^{\text{CTM}}(x_j^{(r)}, t_i^{(k)}), & \text{align,} \\ D_{r,k}[i - 1, j] + c_{\text{missing}}^{\text{CTM}}(t_i^{(k)}), & \text{missing,} \\ D_{r,k}[i, j - 1] + c_{\text{skip}}^{\text{CTM}}(x_j^{(r)}), & \text{skip.} \end{cases} \tag{6}$$

Trong đó, *align* tiêu thụ đồng thời một token OCR và một vị trí mẫu; *missing* đánh dấu vị trí mẫu không có quan sát tương ứng; còn *skip* loại một token OCR dư. CTM chỉ dùng các thao tác này để căn vị trí, không tự thay thế ký tự mơ hồ. Bộ chi phí

căn chỉnh của CTM là

$$c_{\text{align}}^{\text{CTM}}(x, t) = \begin{cases} 0, & t \in \{D, L\}, x \models t, \\ 0, & t \notin \{D, L, A\}, x = t, \\ 0.1, & t = A, x \models t, \\ 0.45, & t \in \{D, L, A\}, x \text{ là chữ hoặc số nhưng } x \not\models t, \\ 2.0, & \text{các trường hợp còn lại,} \end{cases} \quad (7)$$

$$c_{\text{missing}}^{\text{CTM}}(t) = \begin{cases} 1.2, & t \in \{D, L, A\}, \\ 1.0, & t = [\text{SEP}], \\ 0.2, & t = ., \\ 0.75, & t = -, \end{cases} \quad (8)$$

$$c_{\text{skip}}^{\text{CTM}}(x) = \begin{cases} 1.0, & x \text{ là chữ hoặc số,} \\ 0.65, & x \text{ là token phân cách.} \end{cases}$$

Chi phí 0 chỉ xuất hiện khi ký tự đã đúng loại slot D hoặc L , hoặc khi token phân cách trùng chính xác với mẫu. Slot hỗn hợp A nhận chi phí 0.1 để ưu tiên các mẫu có ràng buộc chữ–số cụ thể hơn. Chi phí 0.45 cho phép đường căn chỉnh tiếp tục khi gặp ký tự sai loại slot, nhưng ký tự đó không được đưa vào biểu quyết sau khi truy vết.

Với mỗi chuỗi, hệ thống tính chi phí cuối $D_{r,k}[n_k, m_r]$ cho mọi mẫu.

Sau khi có $\mathcal{T}^*$, từng chuỗi được căn lại với mẫu này bằng Công thức (6). Hệ thống truy vết từ $D_{r,*}[n_*, m_r]$ về $D_{r,*}[0, 0]$ để xác định token nào thuộc từng vị trí mẫu. Chỉ các ký tự đúng loại slot mới được giữ lại; vị trí bị thiếu hoặc căn với ký tự sai loại được xem là không có quan sát. Các dấu phân cách không tham gia biểu quyết mà được khôi phục trực tiếp từ mẫu chung với độ tin cậy quy ước là 0.90.

Biểu quyết theo slot đã căn chỉnh. Gọi $a_{r,j}$ và $p_{r,j}$ lần lượt là ký tự và xác suất của quan sát hợp lệ tại slot j sau khi căn chuỗi thứ r . Tập chỉ số các chuỗi có quan sát hợp lệ tại slot này là

$$\mathcal{V}_j = \{r \mid a_{r,j} \neq \emptyset\}. \quad (9)$$

Với mỗi ký tự c xuất hiện tại slot j , số phiếu, tỷ lệ hỗ trợ và độ tin cậy trung bình

được tính như sau:

$$\begin{aligned}
v_j(c) &= \sum_{r \in \mathcal{V}_j} \mathbb{I}(a_{r,j} = c), \\
\rho_j(c) &= \frac{v_j(c)}{|\mathcal{V}_j|}, \\
\bar{p}_j(c) &= \frac{1}{v_j(c)} \sum_{r \in \mathcal{V}_j} \mathbb{I}(a_{r,j} = c) p_{r,j}.
\end{aligned} \tag{10}$$

Ký tự có số phiếu lớn nhất được chọn làm ứng viên c_j^* ; nếu nhiều ký tự có cùng số phiếu, tổng độ tin cậy được dùng để phá hòa. Ứng viên chỉ được chấp nhận khi đồng thời thỏa mãn

$$\rho_j(c_j^*) \geq \tau_{\text{vote}} = 0.5, \quad \bar{p}_j(c_j^*) \geq \tau_{\text{conf}} = 0.50. \tag{11}$$

Mẫu số của ρ_j là số quan sát hợp lệ tại slot j , không phải tổng số khung hình trong bộ đệm. Tương tự, $\bar{p}_j$ chỉ được tính trên các quan sát bỏ phiếu cho ký tự thắng. Nếu $\mathcal{V}_j$ rỗng hoặc một trong hai điều kiện ở Công thức (11) không đạt, slot được gán ký hiệu “?”. Track chỉ được chấp nhận khi không còn slot vô định và chuỗi hoàn chỉnh vượt qua bước xác thực định dạng biến số.